Resumen los métodos y pasos en Java para conectar a una base de datos PostgreSQL mediante JDBC:

---

1. Importar el driver JDBC de PostgreSQL

Antes de todo, asegúrate de tener el driver JDBC en tu proyecto:

- **Si usas Maven, añade esta dependencia al pom.xml:**

<dependency>

<groupId>org.postgresql</groupId>

<artifactId>postgresql</artifactId>

<version>42.7.3</version>

</dependency>

---

2. Establecer la conexión

a) Cargar el driver (opcional en versiones modernas)

Desde JDBC 4.0, esto se hace automáticamente, pero puede hacerse manualmente:

```
Class.forName("org.postgresql.Driver");
```

b) Crear la conexión

Se utiliza el método **`DriverManager.getConnection()`**, indicando la URL, usuario y contraseña:

```
String url = "jdbc:postgresql://localhost:5432/mi_base";
String user = "postgres";
String password = "1234";

Connection conn = DriverManager.getConnection(url, user, password);
```

3. Ejecutar consultas

Una vez obtenida la conexión (`Connection`), puedes crear objetos para enviar consultas SQL:

a) `Statement`

Para consultas **simples y sin parámetros**:

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM clientes");

while (rs.next()) {
    System.out.println(rs.getString("nombre"));
}
```

b) `PreparedStatement`

Para consultas **con parámetros** (más seguras y eficientes):

```
String sql = "INSERT INTO clientes(nombre, email) VALUES (?, ?)";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, "Juan");
ps.setString(2, "juan@mail.com");
ps.executeUpdate();
```

c) `CallableStatement`

Para ejecutar **procedimientos almacenados**:

```
CallableStatement cs = conn.prepareCall("{call actualizar_cuota(?)}");
cs.setInt(1, 10);
cs.execute();
```

4. Cerrar recursos

Siempre debes cerrar los objetos en orden inverso al que los abriste:

```
rs.close();
stmt.close();
conn.close();
```

💡 Consejo: Usa **try-with-resources** para cerrarlos automáticamente:

```
try (Connection conn = DriverManager.getConnection(url, user,
password);
     PreparedStatement ps = conn.prepareStatement("SELECT * FROM
clientes");
     ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }
}
```

5. Métodos más usados

Clase	Método	Descripción
`DriverManager`	`getConnection(String url, String user, String password)`	Crea la conexión con la base de datos
`Connection`	`createStatement()` / `prepareStatement(String sql)`	Crea un objeto para ejecutar SQL
`Statement`	`executeQuery(String sql)`	Ejecuta una consulta SELECT
`Statement`	`executeUpdate(String sql)`	Ejecuta INSERT, UPDATE o DELETE
`PreparedStatement`	`setString()`, `setInt()`, etc.	Asigna valores a los parámetros
`ResultSet`	`next()`, `getString()`, `getInt()`, etc.	Lee los resultados de una consulta
`Connection`	`close()`	Cierra la conexión

6. Ejemplo Práctico

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ConexionPostgres {
  public static void main(String[] args) {

    // 1️⃣ Datos de conexión
    String url = "jdbc:postgresql://localhost:5432/mi_base";
    String user = "postgres";
    String password = "1234";
```

```
    // 2️⃣ Bloque try-with-resources para cerrar automáticamente recursos
    try (Connection conn = DriverManager.getConnection(url, user, password)) {
      System.out.println("✅ Conexión establecida correctamente.");

      // 3️⃣ INSERTAR un registro
      String insertSQL = "INSERT INTO clientes (nombre, apellidos, email, teléfono,
estado, idCuota) VALUES (?, ?, ?, ?, ?, ?)";
      try (PreparedStatement ps = conn.prepareStatement(insertSQL)) {
        ps.setString(1, "Juan");
        ps.setString(2, "Pérez García");
        ps.setString(3, "juan.perez@example.com");
        ps.setString(4, "600123456");
        ps.setString(5, "activo");
        ps.setInt(6, 1); // idCuota = 1, por ejemplo
        int filas = ps.executeUpdate();
        System.out.println("🟢 " + filas + " fila(s) insertada(s).");
      }

      // 4️⃣ CONSULTAR registros
      String querySQL = "SELECT idCliente, nombre, apellidos, email FROM
clientes";
      try (PreparedStatement ps = conn.prepareStatement(querySQL);
           ResultSet rs = ps.executeQuery()) {

        System.out.println("\n📋 Lista de clientes:");
        while (rs.next()) {
          int id = rs.getInt("idCliente");
          String nombre = rs.getString("nombre");
          String apellidos = rs.getString("apellidos");
          String email = rs.getString("email");
          System.out.println(id + " - " + nombre + " " + apellidos + " (" + email + ")");
        }
      }

    } catch (SQLException e) {
      System.out.println("❌ Error de conexión o consulta: " + e.getMessage());
    }
  }
}
```

Explicación paso a paso

1. **Importación de librerías JDBC**
   Se importan las clases necesarias para manejar conexión, sentencias y resultados (Connection, PreparedStatement, ResultSet).

2. **Conexión con la base de datos**
   DriverManager.getConnection() crea la conexión usando la URL JDBC:

3. jdbc:postgresql://<host>:<puerto>/<nombre_base>

Ejemplo: jdbc:postgresql://localhost:5432/mi_base

4. **Inserción con parámetros (PreparedStatement)**
   Se usan los métodos setString(), setInt(), etc., para asignar valores a los placeholders (?).

5. **Consulta y lectura de resultados (ResultSet)**
   rs.next() avanza por cada fila y se leen las columnas con getString(), getInt(), etc.

6. **Cierre automático de recursos**
   Gracias al try-with-resources, se cierran Connection, PreparedStatement y ResultSet sin usar close() manualmente.

Resumen de los métodos JDBC necesarios para ejecutar sentencias SQL DDL (Data Definition Language)

Es decir, aquellas que **definen o modifican la estructura** de la base de datos:
👉 CREATE, ALTER, DROP, TRUNCATE, etc.

---

1. Conexión a la base de datos

Primero se establece la conexión con **DriverManager.getConnection()**, igual que con cualquier operación JDBC:

String url = "jdbc:postgresql://localhost:5432/mi_base";

String user = "postgres";

String password = "1234";

Connection conn = DriverManager.getConnection(url, user, password);

---

2. Creación de la sentencia

Las sentencias **DDL** se ejecutan normalmente con un **Statement** o **PreparedStatement**, pero a diferencia de las consultas **DML**, **no devuelven resultados** (no hay ResultSet).

a) Usando Statement

Ideal para comandos **sin parámetros**:

Statement stmt = conn.createStatement();

stmt.executeUpdate("CREATE TABLE clientes (id SERIAL PRIMARY KEY, nombre VARCHAR(50))");

b) Usando PreparedStatement

Si necesitas generar la sentencia dinámicamente con parámetros:

String sql = "CREATE TABLE IF NOT EXISTS " + "tabla_" + "productos" + " (id SERIAL, nombre VARCHAR(100))";

PreparedStatement ps = conn.prepareStatement(sql);

ps.executeUpdate();

(Ojo: los parámetros con ? no pueden usarse para nombres de tablas o columnas, solo para valores de datos).

---

3. Métodos principales para ejecutar DDL

Método	Clase	Devuelve	Uso típico
executeUpdate(String sql)	Statement / PreparedStatement	int (número de filas afectadas o 0 en DDL)	Para ejecutar CREATE, ALTER, DROP, TRUNCATE
execute(String sql)	Statement / PreparedStatement	boolean (true si devuelve ResultSet, false si no)	Para ejecutar cualquier tipo de sentencia SQL (DDL, DML o DQL)
addBatch(String sql) + executeBatch()	Statement	int[]	Para ejecutar varias sentencias DDL seguidas

---

4. Ejemplos de uso

✅ **Crear una tabla**

```
try (Connection conn = DriverManager.getConnection(url, user, password);
    Statement stmt = conn.createStatement()) {

  String sql = "CREATE TABLE IF NOT EXISTS empleados (" +
          "id SERIAL PRIMARY KEY, " +
          "nombre VARCHAR(50), " +
          "salario DECIMAL(10,2))";

  stmt.executeUpdate(sql);
```

```
    System.out.println("Tabla creada correctamente.");
}
```

✅ **Modificar una tabla**

```
String sql = "ALTER TABLE empleados ADD COLUMN departamento VARCHAR(50)";
stmt.executeUpdate(sql);
```

✅ **Eliminar una tabla**

```
String sql = "DROP TABLE IF EXISTS empleados";
stmt.executeUpdate(sql);
```

✅ **Ejecutar varias sentencias DDL a la vez**

```
stmt.addBatch("CREATE TABLE departamentos (id SERIAL PRIMARY KEY, nombre VARCHAR(50))");
stmt.addBatch("ALTER TABLE empleados ADD COLUMN idDepartamento INT REFERENCES departamentos(id)");
stmt.executeBatch();
```

---

5. Buenas prácticas

- Usar **IF EXISTS / IF NOT EXISTS** para evitar errores si la tabla ya existe o no.
- En operaciones críticas, usar **transacciones** con conn.setAutoCommit(false) y conn.commit().
- Cerrar siempre los recursos (Statement, Connection) o usar **try-with-resources**.
- No usar PreparedStatement con parámetros para nombres de tablas o columnas (solo para valores).

Ejemplo completo y funcional en Java que muestra cómo crear una base de datos PostgreSQL y sus tablas usando JDBC.

Este ejemplo cubre **todo el proceso**, desde conectarse al servidor hasta crear la base de datos y las tablas relacionadas.

```
import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.Statement;

import java.sql.SQLException;

public class CrearBaseDatosGimnasio {

    public static void main(String[] args) {



        // 1 Datos del servidor (sin especificar la base de datos)

        String urlServidor = "jdbc:postgresql://localhost:5432/";

        String usuario = "postgres";

        String password = "1234";

        String nombreBD = "gimnasio_db";



        try {

            // 2 Conexión inicial al servidor PostgreSQL

            try (Connection conn = DriverManager.getConnection(urlServidor, usuario, password);

                 Statement stmt = conn.createStatement()) {
```

```
            // Crear la base de datos si no existe
            String sqlCrearBD = "CREATE DATABASE " + nombreBD;
            stmt.executeUpdate(sqlCrearBD);
            System.out.println("✅ Base de datos '" + nombreBD + "' creada correctamente.");
        } catch (SQLException e) {
            if (e.getSQLState().equals("42P04")) { // Código SQLState de 'database already exists'
                System.out.println("ℹ La base de datos ya existe, se continuará con las tablas.");
            } else {
                throw e;
            }
        }

        // 3️⃣ Conectarse ahora a la nueva base de datos
        String urlBD = urlServidor + nombreBD;
        try (Connection connBD = DriverManager.getConnection(urlBD, usuario, password);
             Statement stmt = connBD.createStatement()) {

            // 4️⃣ Crear tabla 'cuotas'
            String sqlCuotas = """
                CREATE TABLE IF NOT EXISTS cuotas (
                    idCuota SERIAL PRIMARY KEY,
                    modalidad VARCHAR(20) NOT NULL,
                    cuota DECIMAL(8,2) NOT NULL,
                    situacion VARCHAR(10) DEFAULT 'activo'
```

```
            )
        """;
        stmt.executeUpdate(sqlCuotas);
        System.out.println("✅ Tabla 'cuotas' creada correctamente.");

        // 5️⃣ Crear tabla 'clientes' con clave foránea a 'cuotas'
        String sqlClientes = """
            CREATE TABLE IF NOT EXISTS clientes (
                idCliente SERIAL PRIMARY KEY,
                nombre VARCHAR(50) NOT NULL,
                apellidos VARCHAR(80) NOT NULL,
                email VARCHAR(100) UNIQUE,
                telefono VARCHAR(15),
                estado VARCHAR(10) DEFAULT 'activo',
                idCuota INT REFERENCES cuotas(idCuota)
            )
        """;
        stmt.executeUpdate(sqlClientes);
        System.out.println("✅ Tabla 'clientes' creada correctamente.");

    }

    System.out.println("\n🎉 Base de datos y tablas creadas con éxito.");
```

```
        } catch (SQLException e) {

            System.out.println("❌ Error al crear la base de datos o tablas: " + e.getMessage());

        }

    }

}
```